

Project 2 — Phase 1

Learning Week: LangChain + LangGraph Fundamentals

DevSynt AI Automation Internship · Mentor: Afnan Shoukat · Individual

Overview

This week is **learning-focused** — **there is no coding project this week**. LangChain and LangGraph are genuinely difficult topics, so we're not rushing into building anything yet. Go through each day properly instead of skimming — this is the foundation for everything we build next.

We're covering **both LangChain and LangGraph** because our next mini-project uses an **orchestrator agent** — and that specifically requires LangGraph, not just LangChain. A **quiz on these exact concepts** will follow, so treat this as real learning, not a checkbox.

This Week At a Glance

Day	Focus
Day 1	What is LangChain? Chains vs. raw API calls
Day 2	Prompt Templates & Output Parsers
Day 3	Memory & Tool-Calling (concepts)
Day 4	What is LangGraph? Nodes, edges, state
Day 5	Orchestrator Agents

Daily Breakdown

DAY 1

What is LangChain?

Calling an LLM API directly means sending one prompt and getting one reply back — every time, you write the full prompt yourself, and you get raw text back with no structure. LangChain sits on top of this and gives you reusable building blocks: **prompt templates** (prompts with fill-in-the-blank variables), **chains** (prompt template → LLM → output, connected as one repeatable unit), and easy switching between different LLM providers without rewriting your code.

■ KEY TERMS

LLM wrapper · prompt template · chain · LCEL (LangChain Expression Language — the syntax used to connect pieces with the `|` operator)

■ TRY IT YOURSELF

Using a free LLM key (Gemini/Groq), write one basic chain: a prompt template with one variable, connected to the LLM, and print the response. Compare it to calling the API directly with a plain string.

Resource: <https://python.langchain.com/docs/introduction/>

DAY 2

Prompt Templates & Output Parsers

A prompt template lets you write a prompt once with placeholders (e.g. `{product_name}`) and reuse it with different values. An **output parser** solves a different problem: by default an LLM replies in free-flowing text, hard for code to use reliably. An output parser instructs the LLM to reply in a specific structure (commonly JSON), so your code can read fields directly instead of guessing meaning.

■ KEY TERMS

PromptTemplate · StrOutputParser · JsonOutputParser · structured output · Pydantic models

■ TRY IT YOURSELF

Ask an LLM the same question twice — once with a plain prompt, once asking it to reply only in JSON with specific fields. Notice how much easier the JSON version is to work with in code.

Resource: https://python.langchain.com/docs/concepts/output_parsers/

DAY 3

Memory & Tool-Calling (Concepts)

By default an LLM has no memory — every call is independent. **Memory** means keeping track of the conversation so far and feeding it back into each new call, so the bot can refer to earlier messages. **Tool-calling** means giving the LLM access to functions it can choose to use — a calculator, a search function, a database lookup — instead of guessing an answer from what it already knows. The LLM decides *when* to call a tool and *what* to pass into it.

■ KEY TERMS

conversation memory · chat history · tool/function calling · tool schema

■ TRY IT YOURSELF

No coding needed today if you're short on time — but if you want to go deeper, read one real tool-calling example in the LangChain docs and trace through how the LLM decides which tool to call.

Resource: https://python.langchain.com/docs/concepts/tool_calling/

DAY 4

What is LangGraph? Why do we need it?

A chain follows a **fixed sequence** of steps, always in the same order. An **agent** can look at the situation and decide what to do next — skipping, repeating, or choosing between paths. LangGraph lets you build these decision-making systems as a **graph** instead of a straight line. Core concepts: **node** (one step in the graph), **edge** (connection showing what happens next), **conditional edge** (a decision point based on a node's result), and **state** (the data passed along and updated as it moves through the graph).

■ KEY TERMS

node · edge · conditional edge · state · graph vs. chain

■ TRY IT YOURSELF

Sketch (paper or Mermaid) a 3-node graph for any simple decision process — e.g. "check order status" → conditional edge → "in stock" or "out of stock". No code needed yet, just practice thinking in graph terms.

Resource: https://langchain-ai.github.io/langgraph/concepts/low_level/

An **orchestrator agent** receives a task, figures out what needs to happen, and routes it to the right specialized agent(s) — it usually doesn't do the actual work itself. Think of it like a manager: it doesn't clean the data or write the report itself, it decides who should. This is exactly the pattern our upcoming project will use — an orchestrator agent that receives raw data and decides which agent handles it next.

■ KEY TERMS

orchestrator/router pattern · multi-agent system · shared state between agents · supervisor agent

■ TRY IT YOURSELF

Read through one official LangGraph multi-agent example end-to-end. You don't need to run it — just trace how the orchestrator decides where to send each task.

Resource: https://langchain-ai.github.io/langgraph/tutorials/multi_agent/multi-agent-collaboration/

Free Resources — Full List

- **LangChain docs:** <https://python.langchain.com/docs/introduction/>
- **LangChain Academy (free official course):** <https://academy.langchain.com>
- **LangGraph docs:** <https://langchain-ai.github.io/langgraph/>
- **LangGraph intro course (free):** <https://academy.langchain.com/courses/intro-to-langgraph>
- **LangGraph multi-agent tutorial:**
https://langchain-ai.github.io/langgraph/tutorials/multi_agent/multi-agent-collaboration/
- **Prompt engineering guide:** <https://www.promptingguide.ai>

What to Submit — A Static HTML Learning Page

Instead of a coding project this week, build a single **static HTML page (with CSS)** that showcases what you learned. Think of it as a mini personal learning-summary page — well-designed and organized, not just plain text dumped on a page.

Your page should include a section for each of the following:

- What LangChain is and what a chain does — in your own words
- What prompt templates and output parsers are, with a short example
- What memory and tool-calling mean for an LLM app
- What LangGraph is, and the difference between a chain and an agent
- Core LangGraph concepts: nodes, edges, state, conditional edges
- What an orchestrator agent is, and why our upcoming project needs one
- Any diagrams, code snippets, or visuals that help explain the concepts — styled with CSS, make it look genuinely good

Folder Structure (Follow This Exactly)

Inside your GitHub repository, create a folder named exactly **project2-phase1/** with this structure:

```
project2-phase1/
  ■■■ index.html           your learning showcase page
  ■■■ style.css            your CSS styling
  ■■■ README.md           documentation for this folder
  ■■■ screenshots/
    ■■■ page-preview.png  screenshot of the HTML page running in a browser
```

README.md must include: a one-line description of what this folder contains, how to open/run the HTML page (e.g. "open index.html in any browser"), and a short summary (2-3 sentences) of what you personally learned this week.

Screenshot requirement: take at least one clear screenshot of your HTML page actually open and rendered in a browser, and place it inside the `screenshots/` folder. This is proof the page runs correctly, not just that the code exists.

Keep everything organized and named clearly — this folder should look professional, like something you'd be comfortable showing in a portfolio or to a client.

Once pushed, share your **GitHub repository URL** with me on the portal, same as before.

DEADLINE

Tuesday, 10:00 PM

No coding project this week — take your time to actually understand LangChain and LangGraph properly. A quiz on these exact concepts is coming, so this matters. Stuck? Message Afnan between **2 PM – 10 PM**.